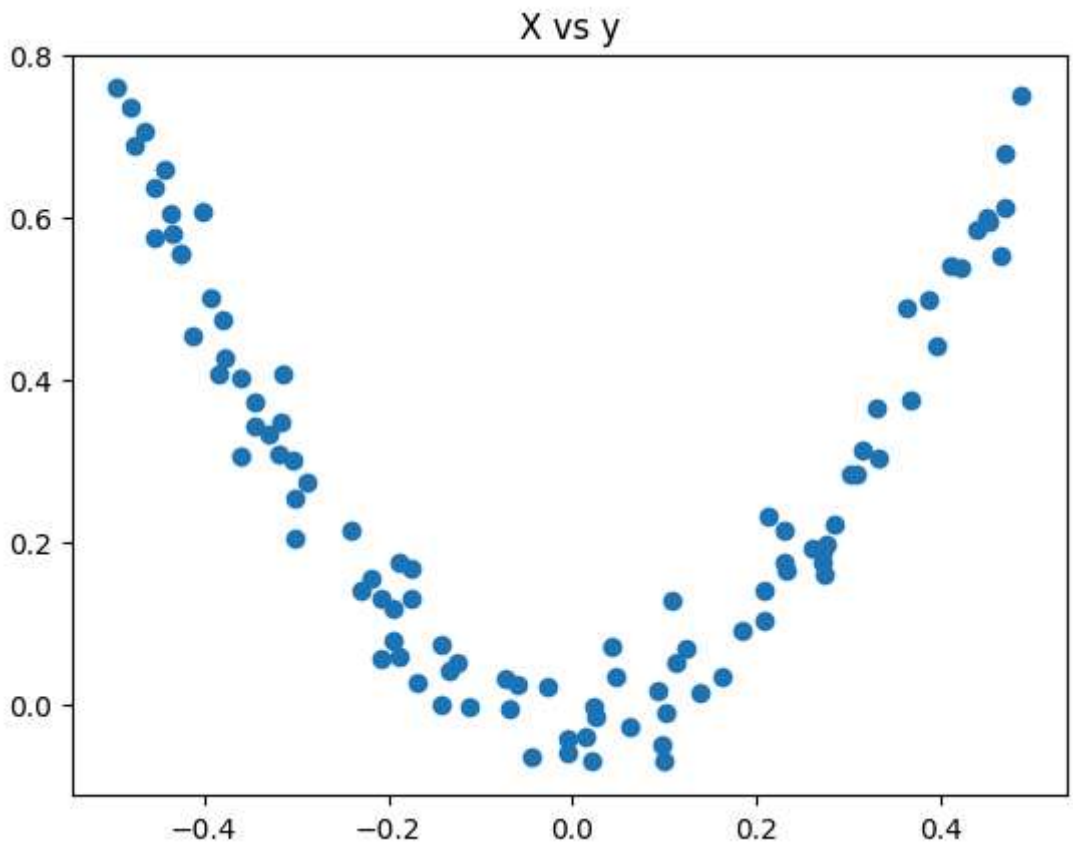
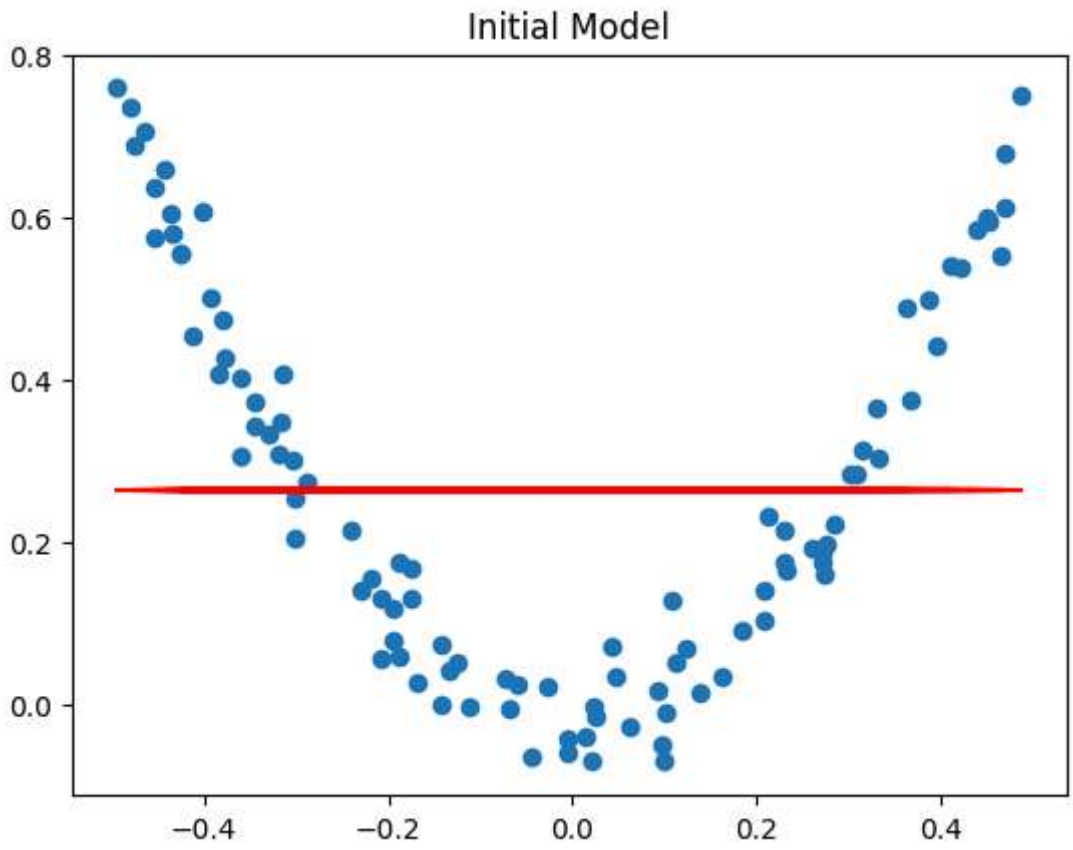
In [12]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.tree import DecisionTreeRegressor, plot_tree

# Create dataset
np.random.seed(42)

X = np.random.rand(100,1) - 0.5
y = 3 * X[:,0]**2 + 0.05 * np.random.randn(100)

# DataFrame
df = pd.DataFrame()
df['X'] = X.reshape(100)
df['y'] = y

print(df)
# Scatter plot
plt.scatter(df['X'], df['y'])
plt.title("X vs y")
plt.show()
```

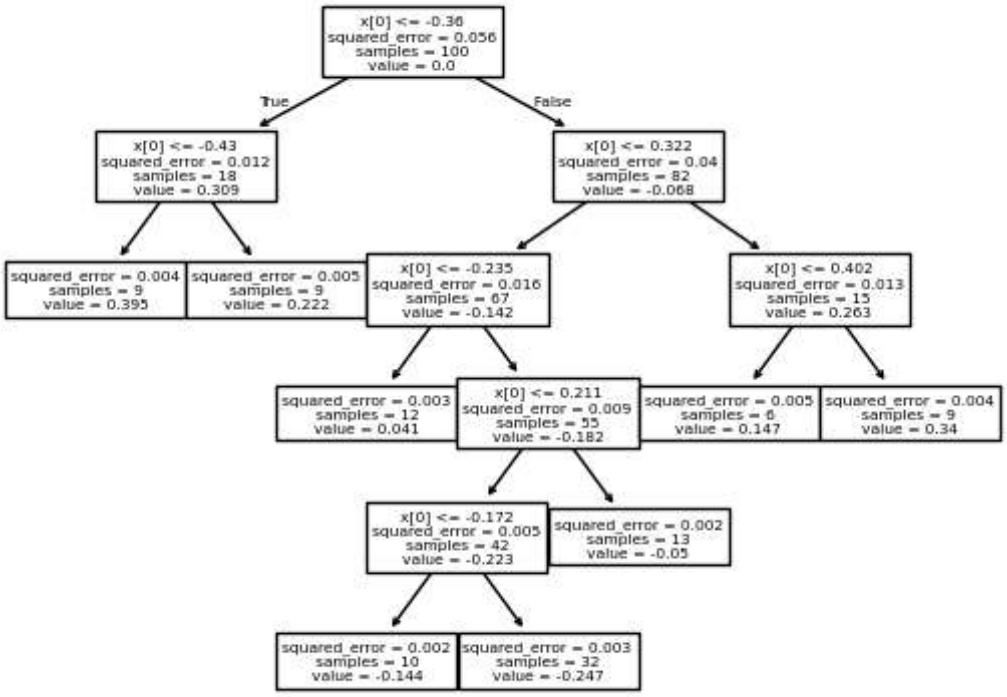
```
          X         y
0  -0.125460  0.051573
1   0.450714  0.594480
2   0.231994  0.166052
3   0.098658 -0.070178
4  -0.343981  0.343986
..       ...       ...
95 -0.006204 -0.040675
96  0.022733 -0.002305
97 -0.072459  0.032809
98 -0.474581  0.689516
99 -0.392109  0.502607

[100 rows x 2 columns]
```



In [13]:
```
# First prediction (mean model)
df['pred1'] = df['y'].mean()

# Residuals
df['res1'] = df['y'] - df['pred1']

# Plot first model
plt.scatter(df['X'], df['y'])
plt.plot(df['X'], df['pred1'], color='red')
plt.title("Initial Model")
plt.show()
```



In [14]:
```
# Train first decision tree on residuals
tree1 = DecisionTreeRegressor(max_leaf_nodes=8)
tree1.fit(df['X'].values.reshape(100,1), df['res1'].values)

plot_tree(tree1)
plt.show()
```



In [17]:
```
# Second prediction
df['pred2'] = df['pred1'] + tree1.predict(df['X'].values.reshape(100,1))

# Residuals again
df['res2'] = df['y'] - df['pred2']

print(df.head())
```

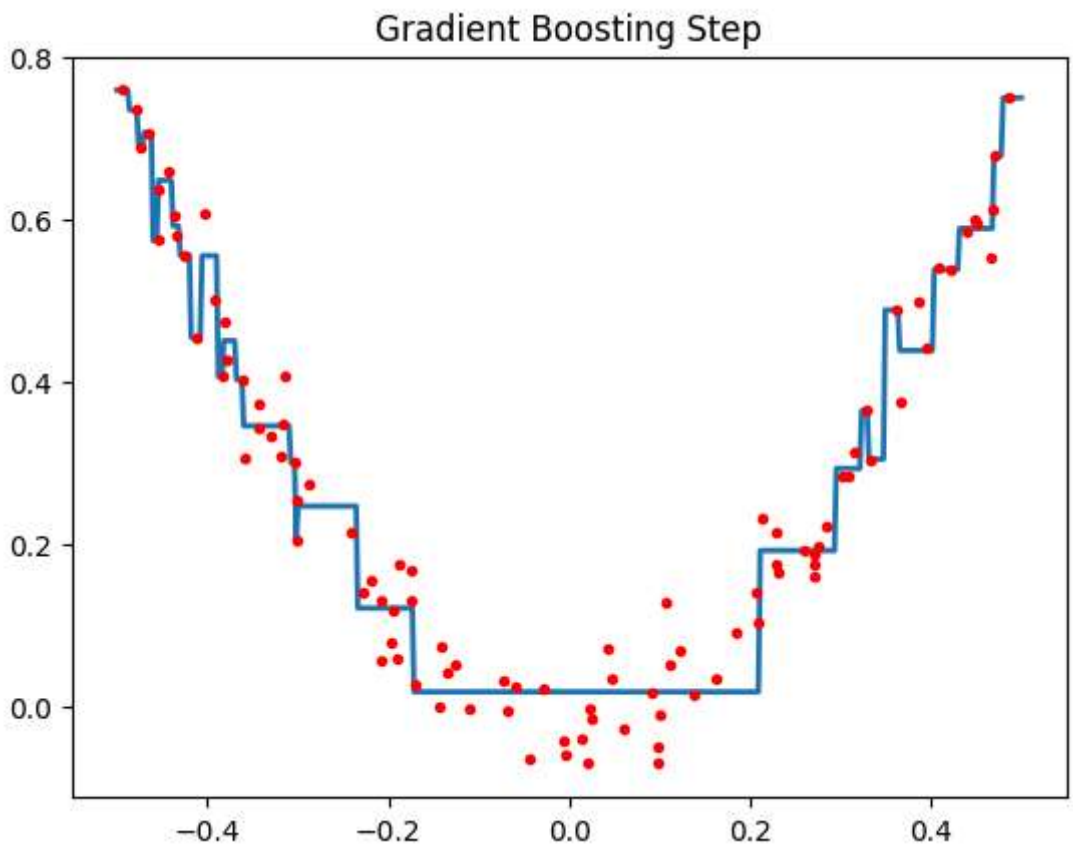
```
          X         y     pred1      res1     pred2      res2
0 -0.125460  0.051573  0.265458 -0.213885  0.018320  0.033253
1  0.450714  0.594480  0.265458  0.329021  0.605884 -0.011404
2  0.231994  0.166052  0.265458 -0.099407  0.215784 -0.049732
3  0.098658 -0.070178  0.265458 -0.335636  0.018320 -0.088497
4 -0.343981  0.343986  0.265458  0.078528  0.305965  0.038021
```

In [21]:
```
def gradient_boost(X, y, number, lr, count=1, regs=[], foo=None):

    if number == 0:
        return

    # Update residuals
    if count > 1:
        y = y - regs[-1].predict(X)
    else:
        foo = y

    # Train tree
    tree_reg = DecisionTreeRegressor(max_depth=5, random_state=42)
    tree_reg.fit(X, y)

    regs.append(tree_reg)
```

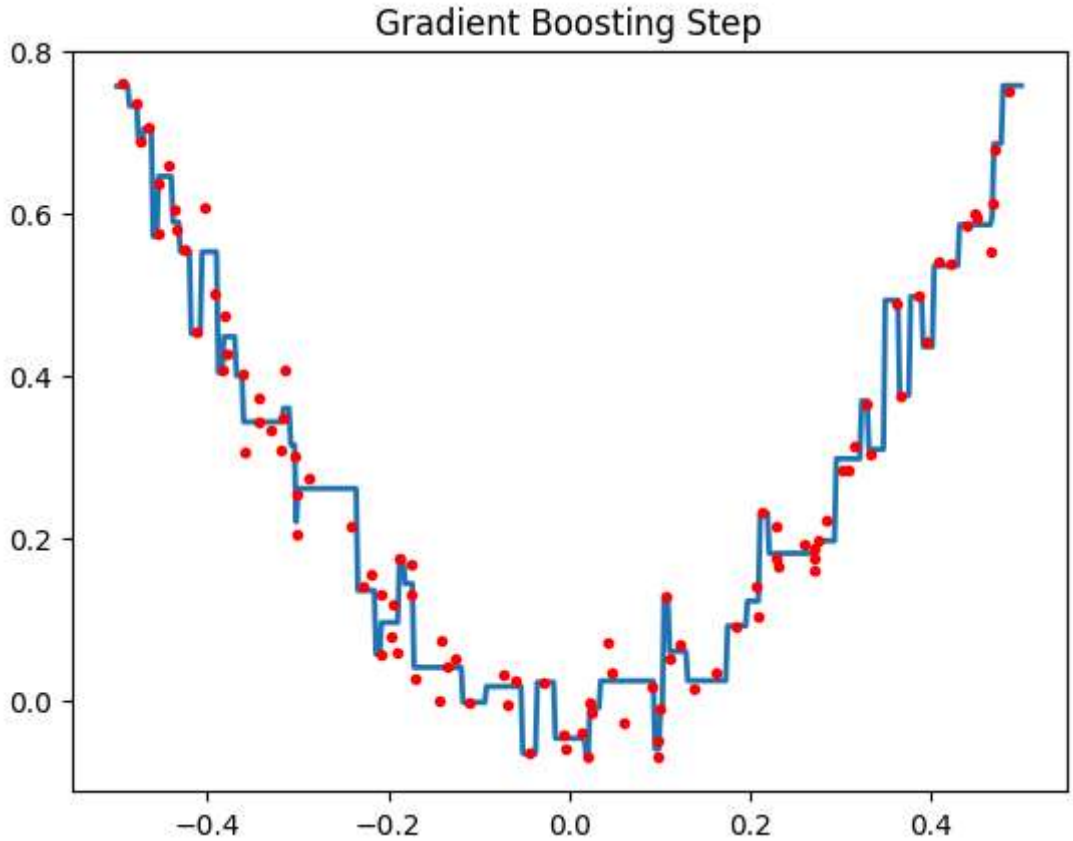
```
    # Prediction
    x1 = np.linspace(-0.5, 0.5, 500)
    y_pred = sum(lr * reg.predict(x1.reshape(-1,1)) for reg in regs)

    print("Iteration:", number)

    plt.figure()
    plt.plot(x1, y_pred, linewidth=2)
    plt.plot(X[:,0], foo, "r.")
    plt.title("Gradient Boosting Step")
    plt.show()

    gradient_boost(X, y, number-1, lr, count+1, regs, foo)
```

In [22]:
```
np.random.seed(42)
X = np.random.rand(100,1) - 0.5
y = 3 * X[:,0]**2 + 0.05 * np.random.randn(100)

gradient_boost(X, y, number=5, lr=1)
```

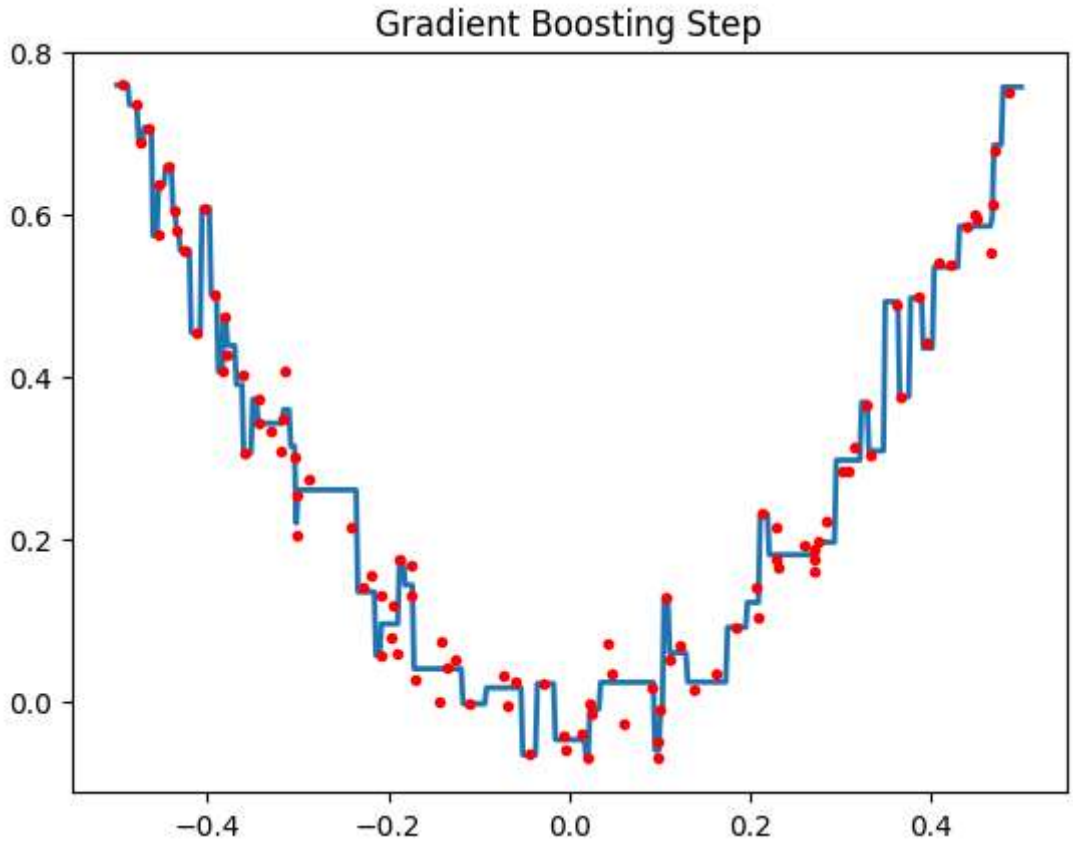
```
Iteration: 5
```



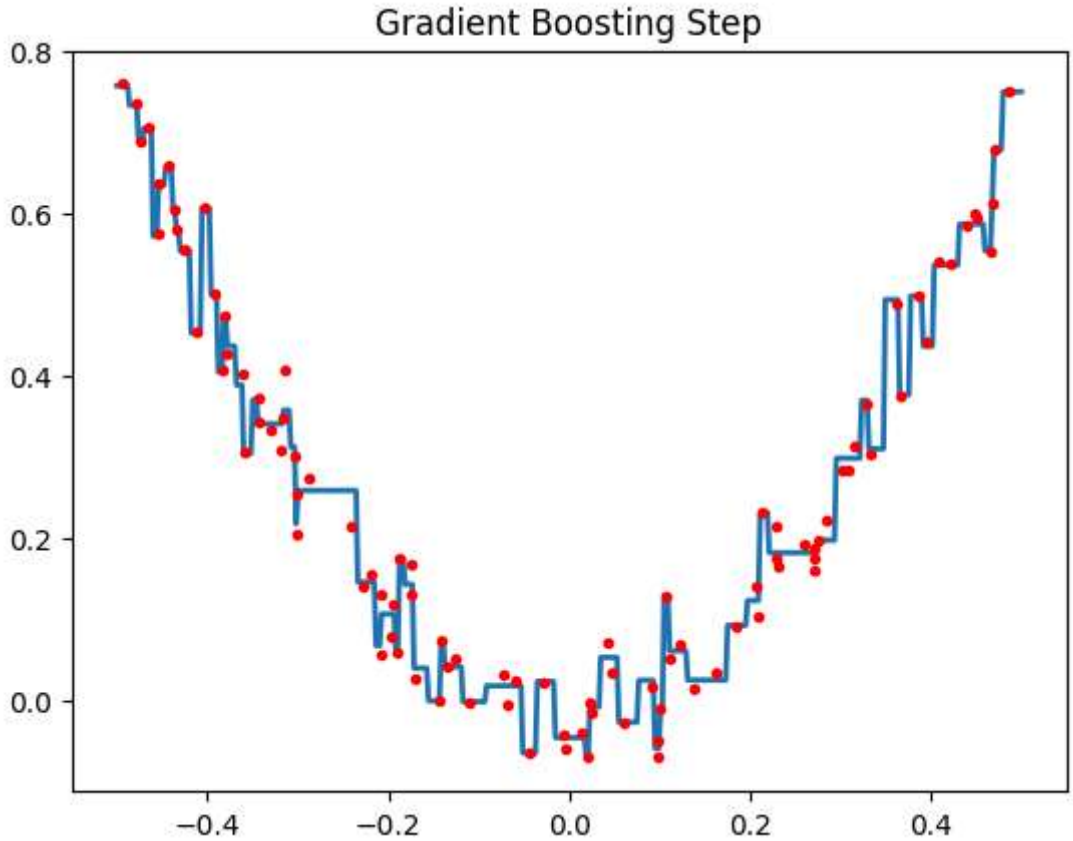

```
Iteration: 4
```



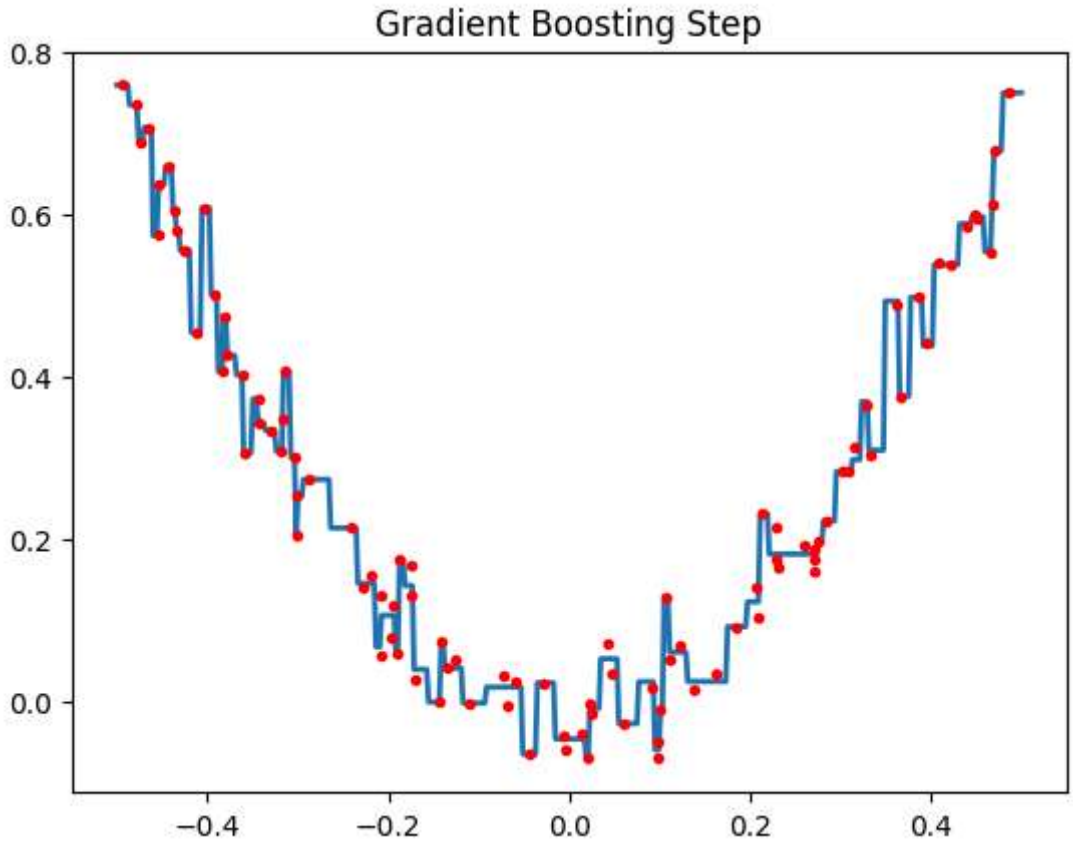

```
Iteration: 3
```




```
Iteration: 2
```




```
Iteration: 1
```




```
In [ ]:
```